Module 4: Modular and Recursive Programming

Premanand S.

Assistant Professor
School of Electronics Engineering
Vellore Institute of Technology, Chennai Campus

premanand.s@vit.ac.in

September 20, 2025

What is Modular Programming?

- Breaking a program into **small, independent, reusable modules/functions**.
- Each module performs a **specific task**.
- **Analogy:** A car has engine, wheels, brakes – each is a module working together.

Why Modular Programming?

- **Reusability:** Write once, use multiple times.
- **Readability:** Smaller code chunks are easier to understand.
- **Maintainability:** Fix or upgrade one module without affecting the whole program.
- **Debugging:** Errors are easier to locate in smaller modules.

Monolithic vs Modular Code

Feature	**Monolithic Code**	**Modular Code**
Structure	Single large block	Divided into smaller modules
Reusability	Low	High
Readability	Hard	Easy
Maintenance	Hard	Easy
Debugging	Difficult	Easier

Example: Adding Two Numbers (Monolithic)

```
a = int(input("Enter first number: "))
b = int(input("Enter second number: "))
sum = a + b
print("Sum:", sum)
```

Example: Adding Two Numbers (Modular)

```
def add_numbers(x, y):
    return x + y

num1 = int(input("Enter first number: "))
num2 = int(input("Enter second number: "))
result = add_numbers(num1, num2)
print("Sum:", result)
```

Real-world Analogy

- **Restaurant Kitchen:**
  - Chef A $\rightarrow$ Prepares appetizers
  - Chef B $\rightarrow$ Prepares main course
  - Chef C $\rightarrow$ Prepares desserts
- Each chef (module) works independently but contributes to the final meal (program).

Functions in Python

- Functions are **blocks of code** designed to perform a specific task.
- They promote **reusability**, **readability**, and **modularity**.
- Defined using the keyword `def`.

Function Definition and Call

```
def greet():
    print("Hello, Welcome to Python!")

# Function call
greet()
```

Output: Hello, Welcome to Python!

Return vs Print

```
def add(a, b):
    return a + b  # returns value

result = add(3, 5)
print("Sum:", result)   # prints 8
```

- `return` sends a value back to the caller.
- `print` only displays the value, cannot be reused.

Positional Arguments

```
def multiply(x, y):
    return x * y

print(multiply(3, 4))  # 12
```

Keyword Arguments

```
def introduce(name, age):
    print(f"My name is {name}, I am {age} years old")

introduce(age=21, name="Python")
```

Default Arguments

```
def greet(name="Guest"):
    print("Hello", name)

greet()             # Hello Guest
greet("Rahul")      # Hello Rahul
```

Arbitrary Arguments

```
def add_all(*args):
    return sum(args)

print(add_all(1,2,3,4))  # 10

def show_details(**kwargs):
    for k, v in kwargs.items():
        print(k, ":", v)

show_details(name="Python", age=21)
```

Local vs Global Variables

```
x = 10  # global

def func():
    x = 5  # local
    print("Inside:", x)

func()
print("Outside:", x)
```

Output: Inside: 5, Outside: 10

Using global Keyword

```
count = 0

def increment():
    global count
    count += 1

increment()
print(count)  # 1
```

Function Basics

1. Write a function `greet_user(name)` that prints a greeting.
2. Modify it to return the greeting instead of printing.
3. Write a function `sum_of_digits(n)` that returns the sum of digits of a number.
4. Write a function `is_divisible(a, b)` that returns `True` if a is divisible by b, else `False`.

Return Values vs Print

1. Write a function that calculates the area of a rectangle:
   - First version: prints the area.
   - Second version: returns the area.
2. Which approach is more reusable? Why?
3. Can a function return multiple values?
   - Example: `divide(a, b)` returns quotient and remainder.

Argument Types

1. Write `student_info(name, age, grade="Not Provided")` and call it in different ways:
   - Only name and age
   - All three arguments
   - Using keyword arguments in any order
2. Write `shopping_total(*prices)` that returns total of any number of prices.
3. Write `person_details(**kwargs)` that prints key-value pairs.
4. Write `mix_args(a, b=5, *args, **kwargs)` and experiment with different calls.

Scope of Variables

1. Define a global variable `count=0`.
2. Write a function `increment()` that increases it by 1 each time.
3. Why does it fail without the `global` keyword?
4. Write a function where a local variable has the same name as a global variable.
5. Discussion: Why should global variables be minimized?

Applied Function Problems

1. `grade_marks(marks)` → average, highest, lowest.
2. `bank_account(balance, transaction, amount=0)`
   - deposit → add amount
   - withdraw → subtract if enough balance
3. `word_count(text)` → frequency of each word.
4. `shopping_cart(*items)` → prints number of items and list.
5. `employee_salary(base, bonus=0, tax=10)` → net salary after bonus and tax.
6. `menu()` → shows options and calls correct function.